QueryAPIGate HOW-TO GUIDE 61 · START

Connect to Trino

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=61-connect-to-trino

---

Time: 10 minutes. **You'll end up with:** one connection that reaches every catalog behind a Trino endpoint - Hive, Iceberg, Delta, Kafka, object stores, other databases - each one a governed, saved REST and MCP query, with `allowed_tables`, masking and a real query time limit.

> **Experimental** (ADR 0026). What it covers, the catalog model and its limits: Trino connections. Before this, Trino was reachable only through generic JDBC - no time limit, no schema browser, no `allowed_tables` or masking. The native type fixes all of that.

Trino is a *federation* engine: one endpoint, many catalogs, each a different underlying system. That one fact shapes everything below - most of all how `allowed_tables` works.

Step 1: Add the connection

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "warehouse", "db": "trino",
  "host": "trino.internal", "port": 8080, "user": "queryapigate",
  "database": "tpch",
  "options": { "schema": "tiny", "http_scheme": "http" },
  "active": true
}'
```

Needs `pip install "queryapigate[trino]"`. The fields that are specific to Trino:

- **`database` is the Trino *catalog*** (here `tpch`, a catalog every Trino ships with). This is the one field that reads differently from the SQL dialects, where `database` is a database on the server.
- **`options.schema`** is the default schema within that catalog (here `tiny`), and **`options.http_scheme`** is `http` or `https`. A `password`, if given, is sent as HTTP basic auth and only over `https`.

Test and browse it like any other connection - the schema browser reads the catalog's `information_schema`, scoped to the default schema:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections/test -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -d '{"db": "trino", "host": "trino.internal", "port": 8080, "user": "queryapigate", "database": "tpch", "options": {"schema": "tiny", "http_scheme": "http"}}'
# {"elapsed_ms": 518.1}

curl http://127.0.0.1:5000/connections/warehouse/schema -H 'X-API-Key: demo-key'
# {"tables": [{"name": "customer", ...}, {"name": "nation", ...}, {"name": "orders", ...}], "truncated": false}
```

Primary/foreign keys and `SHOW CREATE TABLE` DDL aren't shown - Trino has no enforced constraints and three-part names need more handling, the same as for ClickHouse.

Step 2: `database` is a catalog - switch between them

A Trino endpoint usually exposes several catalogs. In the Console, **Connections → (the connection) → switch database** lists them with `SHOW CATALOGS` and switches which one the connection queries:

```shell
curl 'http://127.0.0.1:5000/api/v1/connections/databases' -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -X POST -d '{"db": "trino", "host": "trino.internal", "port": 8080, "user": "queryapigate", "database": "tpch", "options": {"http_scheme": "http"}}'
# {"databases": ["jmx", "memory", "system", "tpcds", "tpch"]}
```

Within a catalog, tables are addressed by `schema.table`, and across catalogs by `catalog.schema.table` - ordinary Trino SQL. Save a query exactly as for any SQL connection (guide 01):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' -d '{
  "name": "nations_by_region", "connection_name": "warehouse",
  "sql": "SELECT n.name FROM nation n JOIN region r ON n.regionkey = r.regionkey WHERE r.name = :region",
  "parameters": {"region": {"type": "str", "required": true, "description": "Region name, e.g. EUROPE"}},
  "author": "you", "description": "Nations in a region", "publish": true
}'

curl 'http://127.0.0.1:5000/q/nations_by_region?region=EUROPE' -H 'X-API-Key: demo-key'
# [{"name": "FRANCE"}, {"name": "GERMANY"}, {"name": "ROMANIA"}, {"name": "RUSSIA"}, {"name": "UNITED KINGDOM"}]
```

Step 3: `allowed_tables` is fully qualified - the federation safety point

For every other dialect, `allowed_tables` matches a **bare** table name. For a federation engine that would be dangerous: a grant of `orders` would match `orders` in *every* catalog the connection can see. So on Trino the match is **fully qualified** - `catalog.schema.table` - with the catalog and schema a reference omits filled in from the connection's defaults.

A grant of `orders` on the `warehouse` connection above (`tpch` / `tiny`) therefore means **`tpch.tiny.orders` only**:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "reporting", "connections": ["warehouse"], "allowed_tables": ["orders"]
}'

# the default catalog + schema's orders: allowed
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: reporting-secret' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT count(*) FROM orders", "connection_name": "warehouse"}'
# [{"_col0": 15000}]

# the SAME table name in another schema or catalog: refused
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: reporting-secret' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT count(*) FROM sf1.orders", "connection_name": "warehouse"}'
# {"error": "This API key may only query these tables: tpch.tiny.orders. Forbidden: tpch.sf1.orders.",
#  "code": "table_not_allowed"}
```

To grant a table in another schema or catalog, qualify the grant: `"allowed_tables": ["sf1.orders"]` for `tpch.sf1.orders`, or `"hive.sales.orders"` for a different catalog entirely. Data-reading table functions (`TABLE(system.query(...))`, `table_changes(...)`) name no table the list can check and are refused the same way.

What governance you get - and what you don't

Everything works through sqlglot's `trino` dialect, so it's the same as any SQL connection:

- **Column masking** (guide 45) and the tables-and-joins diagram both work.
- **The query time limit** cancels the running query on the Trino cluster, so a slow federated query can't run forever.
- **Read-only** rests on the SQL guard (Trino has no read-only session mode), the same as `duckdb` / `h2` / `jdbc`: a non-read statement is refused before it runs.

Not there yet:

- **Agent writes** (guide 49) aren't supported on Trino.
- **Primary/foreign keys and DDL** in the schema browser (Trino has no enforced constraints).

The outer boundary: a scoped Trino user

A connection reaches whatever its Trino **user** can reach. `allowed_tables` narrows *within* that, but the strongest boundary is Trino's own access control: give each connection a **read-only Trino user scoped to the catalogs and schemas it should see**, and let QueryAPIGate's grants narrow from there. That way a mistake in a grant can't reach a catalog the user was never allowed to touch.

Next steps

- Restrict a key to specific tables - the full `allowed_tables` reference; Trino's qualified matching is the one dialect-specific twist.

- Hide personal data from a partner or an AI agent - masking works on Trino like any SQL connection.
- Let an AI agent call your saved queries via MCP - a Trino-backed saved query is an MCP tool like any other.

READ NEXT

See what's being used, from the Console

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=61-connect-to-trino

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate